

FreeRTOS

Table of Contents

1. [What is an RTOS and why do we need one](#)
2. [Tasks — the basic unit of execution](#)
3. [Task priorities and the scheduler](#)
4. [Task states](#)
5. [Queues — passing data between tasks](#)
6. [Semaphores — coordinating access to shared resources](#)
7. [Mutexes — protecting shared data](#)
8. [Task notifications — lightweight signaling](#)
9. [Timers](#)
10. [Memory — stack and heap](#)
11. [ESP32-specific FreeRTOS — dual core](#)
12. [SmartDB task architecture](#)
13. [Common mistakes](#)
14. [Glossary](#)

What is an RTOS

The problem with a simple loop

Most beginner embedded code looks like this:

```
void loop() {  
    read_sensor();           // takes 5ms  
    run_fft();               // takes 15ms  
    send_telemetry();        // takes 10ms (WiFi TX)  
    check_threshold();       // takes 1ms  
    // repeat  
}
```

This works fine when everything is predictable and nothing is time-critical. It breaks down when:

- `send_telemetry()` blocks for 50ms waiting for a WiFi ACK — during which `check_threshold()` never runs and a fault goes undetected
- You need two things to happen simultaneously — like sampling sensors at exactly 100ms while also handling an incoming OVERRIDE command
- One slow operation (DTLS handshake) delays everything else

What an RTOS provides

An RTOS (Real-Time Operating System) is a scheduler — it manages multiple independent units of execution (tasks) and switches between them rapidly, creating the illusion that they run simultaneously.

More importantly, it lets you assign **priorities** — critical tasks (safety layer) always get CPU time before lower-priority tasks (telemetry), regardless of what the lower-priority tasks are doing.

FreeRTOS is the most widely used RTOS for microcontrollers. ESP-IDF includes a customized version that runs on both ESP32 cores.

Tasks

A **task** is a C function that runs as if it has the CPU to itself. In reality, FreeRTOS switches rapidly between tasks — but each task's code is written as if nothing else is running.

Creating a task

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

// Task function — must never return, must loop forever or delete itself
void my_task(void *pvParameters) {
    while (1) {
        // do work here
        vTaskDelay(pdMS_TO_TICKS(100)); // yield CPU for 100ms
    }
}

// Create the task — call this from app_main() or another task
xTaskCreate(
    my_task,           // function to run
    "my_task",        // name (for debugging only)
    4096,             // stack size in bytes
    NULL,             // parameter passed to the task function
    5,                // priority (higher = more important)
    NULL,             // handle — pass a TaskHandle_t* if you need to
    // reference this task later (suspend, delete,
    notify)
);
```

Task function rules

- **Never return** — a task function that returns crashes the system. Always use `while(1)` or call `vTaskDelete(NULL)` when done.
- **Always yield** — a task that never calls `vTaskDelay()` or blocks on a queue/semaphore will starve all lower-priority tasks. The watchdog timer will reset the ESP32 if a task monopolizes the CPU.

Deleting a task

```
vTaskDelete(NULL); // delete the calling task (itself)
vTaskDelete(handle); // delete another task by its handle
```

Priorities and the scheduler

Priority numbers

FreeRTOS priorities are integers — **higher number = higher priority**. On ESP-IDF, valid priority range is 0 (lowest) to `configMAX_PRIORITIES - 1` (highest, typically 25).

The scheduler always runs the highest-priority task that is **ready to run** (not waiting, not blocked). If two tasks have the same priority, they take turns in round-robin fashion.

Preemption

FreeRTOS is **preemptive** — if a high-priority task becomes ready while a low-priority task is running, the scheduler immediately switches to the high-priority task. The low-priority task is paused mid-execution and resumes later exactly where it stopped.

This is critical for SmartDB: when a hard threshold breach occurs, the safety task must preempt everything else immediately — not wait for the current FFT computation or WiFi transmission to finish.

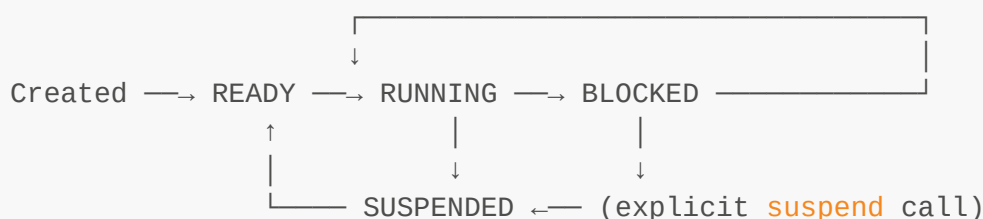
SmartDB priority design

Priority 10	→	safety_layer_task	HIGHEST — must always preempt everything
Priority 7	→	node_fsm_task	Manages state transitions
Priority 5	→	sensor_sampling_task	100ms cycle, time-sensitive
Priority 5	→	dtls_rx_task	Receive incoming messages
Priority 3	→	telemetry_task	Sends data, can be delayed
Priority 3	→	status_led_task	Visual feedback, lowest stakes
Priority 2	→	uart_jetson_task	Frame forwarding to Jetson

Rule of thumb: if task A must never be delayed by task B, A's priority must be strictly higher than B's.

Task states

Every task is in one of five states at any time:



State	Meaning
Ready	Task is ready to run, waiting for CPU
Running	Task is currently executing on a core

State	Meaning
Blocked	Task is waiting for something (delay, queue, semaphore)
Suspended	Task is explicitly paused (vTaskSuspend), won't run until resumed
Deleted	Task has been deleted, memory will be freed

Blocked is the normal "idle" state for most tasks — they block on a delay or queue, freeing the CPU for other tasks. This is correct behavior.

A task should never busy-wait (loop checking a flag repeatedly without blocking) — it wastes CPU and starves other tasks:

```
// WRONG – busy wait, wastes CPU
while (!data_ready) { }

// CORRECT – block on a queue, releases CPU while waiting
xQueueReceive(data_queue, &data, portMAX_DELAY);
```

Queues

A **queue** is a FIFO buffer that safely passes data between tasks. "Safely" means FreeRTOS handles the synchronization — you don't need to worry about one task reading data while another is writing it.

Creating and using a queue

```
#include "freertos/queue.h"

// Define the data type you'll pass through the queue
typedef struct {
    float irms;
    float temperature;
    float anomaly_score;
} sensor_data_t;

// Create a queue that holds up to 10 sensor_data_t items
QueueHandle_t sensor_queue = xQueueCreate(10, sizeof(sensor_data_t));

// --- In the PRODUCER task (sensor_sampling_task) ---
sensor_data_t reading = { .irms = 5.2f, .temperature = 42.1f,
    .anomaly_score = 0.3f };
xQueueSend(sensor_queue, &reading, pdMS_TO_TICKS(10));
// pdMS_TO_TICKS(10) = wait up to 10ms if the queue is full, then give up

// --- In the CONSUMER task (telemetry_task or safety_layer_task) ---
sensor_data_t received;
if (xQueueReceive(sensor_queue, &received, portMAX_DELAY) == pdTRUE) {
    // portMAX_DELAY = wait forever until data arrives (task blocks,
    releases CPU)
```

```
    process_sensor_data(&received);  
}
```

Queue depth sizing

The queue depth (10 in the example) determines how many items can be buffered if the consumer is slower than the producer.

For SmartDB:

- **sensor_queue** depth ~5 — sensor produces at 100ms, telemetry sends at similar rate, small buffer is enough
- **override_queue** depth 1 — override commands should be processed immediately, no buffering needed or desired
- **alert_queue** depth 10 — alerts can queue up briefly if the DTLS transmission is busy

Sending from an ISR (interrupt)

If you need to send to a queue from an interrupt handler (e.g., ADS1115 ALERT pin triggering an interrupt), use the ISR-safe version:

```
BaseType_t xHigherPriorityTaskWoken = pdFALSE;  
xQueueSendFromISR(queue, &data, &xHigherPriorityTaskWoken);  
portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
```

Semaphores

A **semaphore** is a signaling mechanism — one task signals another that something has happened, without passing data.

Think of it as a flag: one task sets the flag, another task waits for it.

Binary semaphore

```
#include "freertos/semphr.h"  
  
SemaphoreHandle_t alert_semaphore = xSemaphoreCreateBinary();  
  
// --- In the task that detects an anomaly ---  
xSemaphoreGive(alert_semaphore); // signal: alert detected  
  
// --- In the task that handles alerts ---  
if (xSemaphoreTake(alert_semaphore, portMAX_DELAY) == pdTRUE) {  
    // alert_semaphore was given – handle the alert  
    send_alert_message();  
}
```

Counting semaphore

Like a binary semaphore but can be given multiple times before being taken. The count represents "how many events are pending."

```
SemaphoreHandle_t event_sem = xSemaphoreCreateCounting(10, 0);
// max count = 10, initial count = 0
```

When to use semaphore vs queue

- **Queue** — when you need to pass data alongside the signal
- **Semaphore** — when you just need to say "event happened," no data needed

For SmartDB, use a semaphore to signal the node_fsm_task that a threshold breach occurred (safety_layer_task gives it, fsm_task takes it to transition to SAFE_MODE). No data needed — just the signal.

Mutexes

A **mutex** (mutual exclusion) prevents two tasks from accessing the same resource simultaneously — for example, two tasks both trying to write to the same UART or SPI bus at the same time.

```
MutexHandle_t i2c_mutex = xSemaphoreCreateMutex();

// --- In any task that uses I2C ---
if (xSemaphoreTake(i2c_mutex, pdMS_TO_TICKS(100)) == pdTRUE) {
    // Safe to use I2C – we have exclusive access
    ads1115_read(&reading);
    xSemaphoreGive(i2c_mutex); // release when done
} else {
    // Could not get mutex in 100ms – log error, skip this cycle
}
```

Mutex vs binary semaphore

They look similar but have a key difference: mutexes implement **priority inheritance** — if a low-priority task holds a mutex that a high-priority task is waiting for, FreeRTOS temporarily raises the low-priority task's priority to prevent **priority inversion**.

Always use a mutex (not a binary semaphore) to protect shared hardware resources like I2C, UART, SPI.

SmartDB shared resources that need mutexes

i2c_mutex	→ protects I2C bus (ADS1115 + MLX90640 share it)
uart_mutex	→ protects UART TX to Jetson Nano
nvs_mutex	→ protects NVS read/write (PSK storage, config)

Task notifications

Task notifications are a lightweight alternative to semaphores for simple signaling between exactly two tasks. Faster and uses less memory than a semaphore.

```
TaskHandle_t safety_task_handle;

xTaskCreate(safety_task, "safety", 2048, NULL, 10, &safety_task_handle);

// --- From any other task, to wake up safety_task ---
xTaskNotify(safety_task_handle, 0, eNoAction);

// --- Inside safety_task, waiting for notification ---
ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
// Blocks here until notified, then continues
```

For SmartDB, task notifications are the right tool for:

- Waking the FSM task when a state-change trigger fires
- Waking the telemetry task when new sensor data is ready to send

Timers

FreeRTOS software timers run a callback function after a period of time, without needing a dedicated task for timing.

```
#include "freertos/timers.h"

// Callback runs when timer expires
void heartbeat_callback(TimerHandle_t xTimer) {
    // send HEARTBEAT 0x03 message
    send_heartbeat();
}

// Create a timer that fires every 30 seconds, auto-reload = pdTRUE
TimerHandle_t heartbeat_timer = xTimerCreate(
    "heartbeat",
    pdMS_TO_TICKS(30000),    // 30 seconds
    pdTRUE,                  // auto-reload (repeating)
    NULL,
    heartbeat_callback
);

xTimerStart(heartbeat_timer, 0);
```

Important limitation

Timer callbacks run in the **timer daemon task** — a shared context for all timers. Callbacks must be short and must not block. If a timer callback needs to do significant work, it should send a notification or queue message to a dedicated task that does the work.

For SmartDB, use timers for: heartbeat scheduling, DTLS retransmission timeouts, watchdog checks on FSM state transitions.

Memory

Stack vs heap

Every task has its own **stack** — a fixed-size memory block for local variables and function call frames. Stack size is specified at task creation (the **4096** parameter in `xTaskCreate`).

Stack overflow is the most common FreeRTOS crash. Symptoms: random crashes, watchdog resets, corrupted data. To detect it:

```
// In sdkconfig, enable stack overflow detection:
// Component config → FreeRTOS → Check for stack overflow
// Set to "Method 2" for reliable detection

// At runtime, check how much stack a task has used:
UBaseType_t remaining = uxTaskGetStackHighWaterMark(NULL);
// 'remaining' is the minimum free stack ever seen – if it's close to 0,
// increase the stack size
```

Stack size guidelines for SmartDB

Task	Suggested stack size	Reason
safety_layer_task	2048 bytes	Simple threshold comparison, minimal stack use
sensor_sampling_task	4096 bytes	FFT computation uses local arrays
dtls_rx_task	8192 bytes	mbedTLS uses significant stack during handshake
telemetry_task	4096 bytes	Packet assembly, DTLS write
node_fsm_task	3072 bytes	State machine logic, minimal recursion
uart_jetson_task	3072 bytes	UART read/write, frame buffering
status_led_task	1024 bytes	Simple GPIO operations

When in doubt, start larger and reduce after measuring high water mark.

Heap

FreeRTOS objects (queues, semaphores, timers, task stacks) are allocated from a shared heap. ESP32 has ~300KB of available heap.

Monitor heap usage:


```
size_t free_heap = esp_get_free_heap_size();
size_t min_heap  = esp_get_minimum_free_heap_size();
// Log these periodically – if min_heap trends toward 0, you have a leak
```

ESP32-specific FreeRTOS — dual core

ESP32 has two cores: **Core 0** (Protocol CPU) and **Core 1** (Application CPU). ESP-IDF's FreeRTOS runs tasks across both cores.

Default behavior

By default, `xTaskCreate()` lets the scheduler place the task on either core. This is usually fine.

Pinning tasks to a specific core

Use `xTaskCreatePinnedToCore()` to force a task to a specific core:

```
xTaskCreatePinnedToCore(
    safety_layer_task,
    "safety",
    2048,
    NULL,
    10,
    NULL,
    1          // core ID: 0 or 1
);
```

SmartDB core assignment

Core 0 (Protocol CPU)	Core 1 (Application CPU)
WiFi stack (reserved)	safety_layer_task
dtls_rx_task	sensor_sampling_task
telemetry_task	node_fsm_task
uart_jetson_task	status_led_task

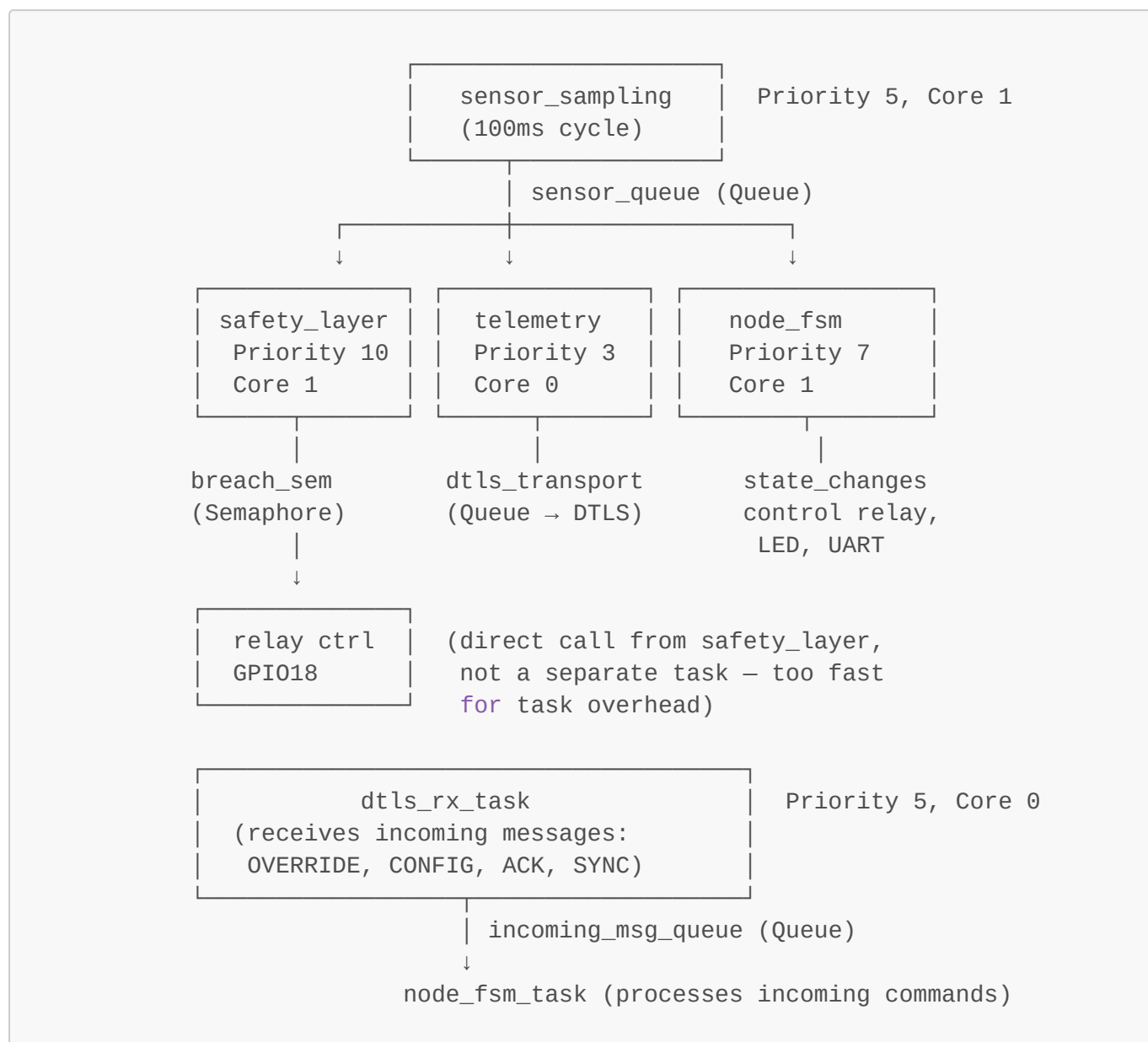
WiFi runs on Core 0 by default in ESP-IDF — placing WiFi-heavy tasks (DTLS, telemetry) on Core 0 reduces inter-core communication overhead. Safety and sensor tasks on Core 1 run independently of WiFi activity — meaning a WiFi hiccup cannot delay the safety layer.

Inter-core communication

Tasks on different cores communicate the same way as tasks on the same core — queues, semaphores, mutexes. FreeRTOS handles the cross-core synchronization transparently.

SmartDB task architecture

Here is the complete task structure for SmartDB firmware, showing how tasks communicate and which FreeRTOS primitives connect them.



Key design decisions reflected here

safety_layer_task calls relay_control directly — not through a queue. Queues introduce latency (the relay must wait for the queue to be processed). A threshold breach must cut the relay within microseconds — a direct function call is the only correct approach.

dtls_rx_task is separate from telemetry_task — incoming and outgoing DTLS traffic are on separate tasks so a slow outgoing transmission doesn't block processing of an incoming OVERRIDE command.

node_fsm_task owns all state transitions — all tasks that detect state-change conditions (safety breach, missing heartbeat ACK, incoming OVERRIDE) communicate with node_fsm via queue or semaphore. The FSM itself applies the transition — no task other than node_fsm should change system state directly.

Common mistakes

1. Forgetting to yield in a task

```
// WRONG — monopolizes CPU, watchdog resets
while (1) {
    check_something();
    // no delay, no blocking call
}

// CORRECT
while (1) {
    check_something();
    vTaskDelay(pdMS_TO_TICKS(10));
}
```

2. Using vTaskDelay in a timer callback Timer callbacks must not block. Use `xTaskNotify` or `xQueueSend` instead.

3. Stack too small Always check `uxTaskGetStackHighWaterMark` during development. A high water mark below 256 bytes is dangerous.

4. Calling FreeRTOS API from ISR without the ISR-safe version Use `xQueueSendFromISR`, `xSemaphoreGiveFromISR`, `xTaskNotifyFromISR` in interrupt handlers — never the non-ISR versions.

5. Priority inversion — using binary semaphore instead of mutex If a shared resource is protected by a binary semaphore instead of a mutex, a low-priority task holding the semaphore can block a high-priority task indefinitely. Always use `xSemaphoreCreateMutex()` for resource protection.

6. Deadlock — two tasks each waiting for the other's mutex

```
Task A holds mutex_1, waiting for mutex_2
Task B holds mutex_2, waiting for mutex_1
→ both blocked forever
```

Avoid by always acquiring mutexes in the same order across all tasks.

7. Modifying a queue handle after task creation `QueueHandle_t` is a pointer. Create all queues before creating tasks that use them — tasks should receive the queue handle as a parameter or access it through a global, not create it themselves.

Glossary

Term	Meaning
Blocking	Task is waiting for an event, releases CPU to other tasks
Context switch	Scheduler saves current task state and switches to another task
Core affinity	Which CPU core a task is pinned to
Deadlock	Two or more tasks each waiting for a resource the other holds

Term	Meaning
DRBG	Deterministic Random Byte Generator
FreeRTOS	Open-source real-time operating system for microcontrollers
Handle	A reference (pointer) to a FreeRTOS object (task, queue, semaphore)
Heap	Shared dynamic memory pool
High water mark	Minimum free stack space ever observed in a task
ISR	Interrupt Service Routine — runs in response to a hardware interrupt
Mutex	Mutual exclusion lock with priority inheritance
Preemption	Scheduler interrupts a running task to run a higher-priority one
Priority inversion	Low-priority task blocking a high-priority task via a shared resource
Queue	FIFO buffer for passing data between tasks safely
Semaphore	Signaling primitive — binary (0/1) or counting
Stack	Per-task memory for local variables and function call frames
Stack overflow	Task exceeds its allocated stack — causes crash or corruption
Task	Independent unit of execution with its own stack and priority
Task notification	Lightweight single-value signal between exactly two tasks
Tick	FreeRTOS time unit — typically 1ms on ESP32
Timer daemon	FreeRTOS task that runs software timer callbacks
Watchdog	Hardware timer that resets the ESP32 if not periodically fed
Yielding	Task voluntarily releases CPU (via delay, queue wait, or semaphore)